Campus Notifications Microservice

System Design Document

Roll No: 22MID0052

Backend Track

Stage 1: REST API Design

As a backend developer working on a campus notification platform, the following REST API endpoints define the core actions that the notification platform supports. These endpoints are designed for a frontend developer colleague to consume when users are logged in.

Core Entities

The notification platform revolves around the following entity: Notification, which contains the following fields:

API Endpoints

Response (200 OK):


```
{
"success": true,
"data": {
"notifications": [
{
"id": "b283218f-ea5a-4b7c-93a9-1f2f240d64b0",
"type": "Placement",
"message": "CSX Corporation hiring",
"timestamp": "2026-04-22 17:51:18",
"isRead": false
}
],
"total": 100,
"page": 1,
"limit": 10
}
}
```


Request Headers:

```
Authorization: Bearer <token>
Content-Type: application/json
```

Response (200 OK):


```
{ "success": true, "message": "Notification marked as read" }
```

Real-Time Notification Mechanism

For real-time notifications, WebSockets (via Socket.IO) are used. When a new notification is created, the server emits a notification:new event to connected clients.

Stage 2: Database Design

Based on the REST API contract defined in Stage 1, PostgreSQL is recommended as the persistent storage solution for the campus notification platform.

Why PostgreSQL?

- ACID compliance ensures data consistency for notifications
- Native UUID support for unique notification IDs
- ENUM types map cleanly to notification_type values
- Excellent indexing capabilities for filtering and pagination
- Strong ecosystem support with Node.js (pg, Prisma, TypeORM)
- Supports JSONB for flexible metadata storage if needed

Database Schema

notifications table:

```
CREATE TYPE notification_type AS ENUM ('Event', 'Result', 'Placement');


CREATE TABLE notifications (
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
type        notification_type NOT NULL,
message     TEXT NOT NULL,
timestamp   TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
is_read     BOOLEAN DEFAULT FALSE,
created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
```

Problems at Scale and Solutions

Problem 1: Slow queries as data grows

As millions of notifications accumulate, queries without indexes will perform full table scans.

Solution: Add targeted indexes

```
CREATE INDEX idx_notifications_type ON notifications(type);
CREATE INDEX idx_notifications_timestamp ON notifications(timestamp DESC);
CREATE INDEX idx_notifications_is_read ON notifications(is_read);
CREATE INDEX idx_notifications_type_timestamp ON notifications(type, timestamp DESC);
```

Problem 2: Pagination performance degrades with OFFSET

Using OFFSET for pagination becomes slow on large datasets as the DB must scan all preceding rows.

Solution: Cursor-based pagination

```
SELECT * FROM notifications
```

```
WHERE timestamp < $1
ORDER BY timestamp DESC
LIMIT $2;
```

Problem 3: Storage growth

Old notifications accumulate indefinitely, consuming storage and slowing queries.

Solution: Archival and partitioning

- Partition notifications table by month using PostgreSQL table partitioning
- Archive notifications older than 6 months to cold storage
- Use pg_partman for automated partition management

Key Queries

Get paginated notifications with type filter:

```
SELECT id, type, message, timestamp, is_read
FROM notifications
WHERE ($1::notification_type IS NULL OR type = $1)
ORDER BY timestamp DESC
LIMIT $2 OFFSET $3;
```

Get all students who received a Placement notification in last 7 days:

```
SELECT DISTINCT student_id
FROM notifications
WHERE notification_type = 'Placement'
AND timestamp >= NOW() - INTERVAL '7 days';
```

Stage 3: Indexing Strategy

A developer on the team suggests adding indexes on every column to be safe. This section evaluates whether that advice is effective and proposes the correct indexing strategy.

Query: Students with Placement notification in last 7 days

```
SELECT DISTINCT student_id
FROM notifications
WHERE notification_type = 'Placement'
AND timestamp >= NOW() - INTERVAL '7 days';
```

This query benefits from the composite index on (type, timestamp DESC) which allows the DB to filter by type and time range efficiently in a single index scan without a full table scan.

Stage 4: Performance Optimisation

The notifications are being fetched on each page load for every student, overwhelming the database and causing poor user experience. The following strategies address this.

Problem

Every page load triggers a fresh DB query for notifications. With thousands of concurrent students this causes:

- High DB connection pool exhaustion
- Slow query response times under load
- Poor user experience with visible loading delays

Solutions and Trade-offs

Strategy 1: Server-Side Caching (Redis)

Cache notification results in Redis with a short TTL (e.g., 30-60 seconds).

```
// Cache key per user + filters
const cacheKey = `notifications:${userId}:${type}:${page}`;
const cached = await redis.get(cacheKey);
if (cached) return JSON.parse(cached);
```

Strategy 2: Pagination

Never fetch all notifications at once. Always paginate with limit/offset or cursor-based pagination to reduce per-query data volume.

Strategy 3: WebSocket Push instead of Polling

Instead of fetching on every page load, push new notifications to clients via WebSockets. Clients only query on initial load; updates come via push.

Recommended Combined Approach

- Use Redis cache for initial page load (TTL: 30s)
- Use cursor-based pagination to avoid OFFSET degradation
- Use WebSocket push for real-time new notification delivery
- Invalidate cache when new notifications are created

Stage 5: Reliable Bulk Notification System

During placement season, the HR clicks 'Notify All' and 50,000 students should get an email and in-app notification simultaneously. This stage analyses the proposed implementation and redesigns it for reliability and performance.

Problems with the Original Implementation

Original pseudocode:

```
function notify_all(student_ids: array, message: string):
for student_id in student_ids:
send_email(student_id, message)   # calls Email API
save_to_db(student_id, message)   # DB insert
push_to_app(student_id, message)  # real-time push
```

Identified Shortcomings:

- Sequential processing of 50,000 students is extremely slow
- If send_email fails at student 200, the remaining 49,800 students get nothing
- No retry mechanism for failed email sends
- Email API and DB operations are tightly coupled — one failure blocks both
- No way to track which notifications succeeded or failed
- Memory pressure from loading all 50,000 IDs at once

Redesigned Solution

Architecture: Message Queue + Worker Pattern

Separate the concerns using a message queue (e.g., BullMQ with Redis or RabbitMQ):

Revised Pseudocode:

```
async function notify_all(student_ids: array, message: string):
# 1. Batch insert all notifications to DB first (atomic)
await db.batch_insert(student_ids, message)

# 2. Enqueue email jobs in batches of 100
for batch in chunks(student_ids, 100):
await emailQueue.addBulk(batch.map(id => ({
data: { student_id: id, message },
opts: { attempts: 3, backoff: { type: 'exponential', delay: 2000 } }
})))

# 3. Push notifications via WebSocket/FCM (best effort)
await pushQueue.addBulk(student_ids.map(id => ({ data: { id, message } })))

return { status: 'queued', total: student_ids.length }
```

Should DB save and email send happen together?

No. They should be decoupled. The DB insert should happen first as the source of truth. The email is a side effect that can be retried independently. Coupling them means a failed email prevents the notification from being saved — which is worse than a delayed email.

Key Design Decisions:

- DB insert is synchronous and must succeed first (source of truth)
- Email sending is asynchronous via queue with 3 retry attempts
- Exponential backoff prevents hammering the email API on failure
- Batch processing (100 at a time) prevents memory overflow
- Dead letter queue captures permanently failed jobs for manual review
- Return 202 Accepted immediately — don't block the HR's request

Stage 6: Priority Inbox Algorithm

The Priority Inbox always displays the top N most important unread notifications first. Priority is determined by a combination of weight (placement > result > event) and recency.

Priority Score Formula

Each notification is scored using a combination of type weight and recency:

```
priorityScore = typeWeight * 1000000 + (timestamp in unix ms)
```

This ensures Placement notifications always outrank Result which outrank Event, and within the same type, more recent ones rank higher.

Algorithm: Min-Heap for Top N

To efficiently maintain the top N notifications as new ones keep arriving, a min-heap of size N is used:

- Time complexity: O(M log N) where M = total notifications, N = top count
- Space complexity: O(N) — only stores N items at a time
- Efficient for continuous insertion of new notifications

Implementation (JavaScript):

```
const TYPE_WEIGHTS = { Placement: 3, Result: 2, Event: 1 };

function getPriorityScore(notification) {
const weight = TYPE_WEIGHTS[notification.type] || 0;
const ts = new Date(notification.timestamp).getTime();
return weight * 1_000_000_000_000 + ts;
}

function getTopNNotifications(notifications, n) {
const unread = notifications.filter(n => !n.isRead);
// Min-heap: maintain only top N by score
const heap = [];

for (const notif of unread) {
const score = getPriorityScore(notif);
heap.push({ ...notif, score });
heap.sort((a, b) => b.score - a.score); // keep sorted
```

```
if (heap.length > n) heap.pop(); // remove lowest
}

return heap.sort((a, b) => b.score - a.score);
}
```

Maintaining Top N Efficiently with New Notifications

When new notifications arrive via WebSocket, the heap is updated incrementally:

- If heap has fewer than N items: add the new notification directly
- If new notification's score > minimum score in heap: replace the minimum
- Otherwise: discard the new notification (it doesn't make top N)

This keeps the operation O(log N) per new notification using a proper min-heap data structure (e.g., using a library like heap.js or implementing a binary heap).

API Endpoint for Priority Inbox

```
GET /api/notifications/priority?n=10&notification_type=Placement

Response:
{
"success": true,
"data": {
"notifications": [ ...top N sorted by priority score ],
"n": 10
}
}
```